

The Decision Boundary: When Is Naïve Bayes Linear?

Deriving the Posterior

Computing $P(y = 1 \mid \mathbf{x})$ under the Naïve Bayes model:

$$\begin{aligned} P(y = 1 \mid \mathbf{x}) &= \frac{P(\mathbf{x} \mid y = 1) P(y = 1)}{P(\mathbf{x} \mid y = 0) P(y = 0) + P(\mathbf{x} \mid y = 1) P(y = 1)} \\ &= \frac{1}{1 + \frac{P(\mathbf{x} \mid y = 0) P(y = 0)}{P(\mathbf{x} \mid y = 1) P(y = 1)}} \end{aligned}$$

Taking the log-odds ratio and substituting the Bernoulli likelihoods:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \log \frac{P(y = 1)}{P(y = 0)} + \sum_{j=1}^d \log \frac{P(x_j \mid y = 1)}{P(x_j \mid y = 0)}$$

For binary features, this simplifies to:

$$= \underbrace{\log \frac{\phi_y}{1 - \phi_y}}_{\theta_0} + \sum_{j=1}^d x_j \underbrace{\log \frac{\phi_{j|1}(1 - \phi_{j|0})}{\phi_{j|0}(1 - \phi_{j|1})}}_{\theta_j} = \theta_0 + \boldsymbol{\theta}^\top \mathbf{x}$$

Naïve Bayes Has a Linear Decision Boundary

With binary features, Naïve Bayes produces a posterior of the form:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\theta_0 + \boldsymbol{\theta}^\top \mathbf{x})$$

where σ is the sigmoid function. The decision boundary $\theta_0 + \boldsymbol{\theta}^\top \mathbf{x} = 0$ is a **hyperplane** — exactly the same functional form as logistic regression!

The *same* form, but **different parameters**: NB derives $\boldsymbol{\theta}$ from the generative model; logistic regression optimises $\boldsymbol{\theta}$ directly to minimise cross-entropy loss.

Comparing All Three Classifiers

It is instructive to see how Naïve Bayes, GDA, and logistic regression all fit into the same generative/discriminative framework.

Three Classifiers: What Each Models

	Naïve Bayes		GDA	Logistic Regression
Type	Generative		Generative	Discriminative
Feature type	Discrete	(binary, counts)	Continuous	Any
Key assumption	Cond. independence		Gaussian class-cond.	None on $\mathbf{x}$
What is modelled	$P(x_j y)$ per feature		$P(\mathbf{x} y)$ jointly	$P(y \mathbf{x})$ directly
Parameters	d per class + prior		$\boldsymbol{\mu}_k, \boldsymbol{\Sigma}, \phi$	$\boldsymbol{\theta} \in \mathbb{R}^{d+1}$
Estimation	Closed-form, one pass		Closed-form, one pass	Gradient descent
Decision boundary	Linear	(binary feat.)	Linear (shared $\boldsymbol{\Sigma}$)	Linear
Posterior form	Sigmoid of linear		Sigmoid of linear	Sigmoid of linear
Small-data perf.	Strong		Strong	Weaker

When Naïve Bayes Outperforms Logistic Regression

Generative vs. Discriminative: The Learning Curve

- **Logistic regression** is asymptotically optimal: with infinite data it converges to the best possible linear classifier.
- **Naïve Bayes** converges faster: it reaches its (lower) asymptotic accuracy with far fewer training examples.
- **Crossover:** on small datasets, Naïve Bayes often wins despite its wrong assumption; on large datasets, logistic regression usually wins.

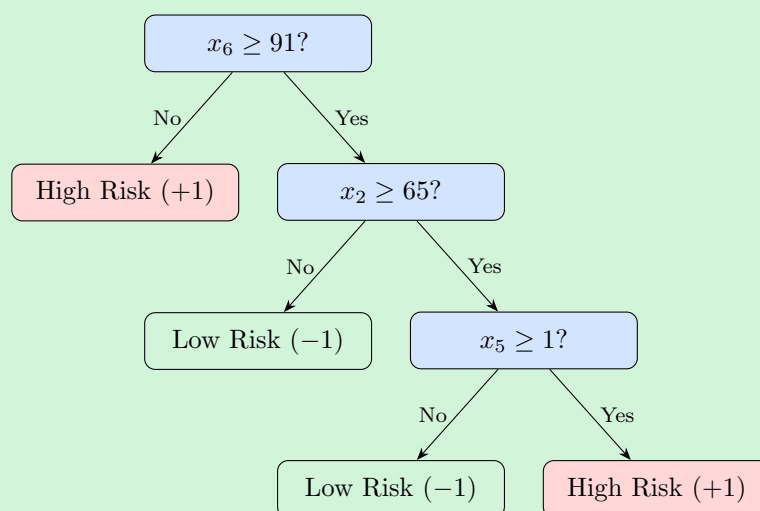
Decision Trees: Examples

1 The Classic Medical Example (Breiman et al., 1984)

Heart-Attack Risk Stratification (Breiman, Friedman, Olshen, Stone, 1984)

Features: x_1 = date, x_2 = age, x_3 = height, x_4 = weight, x_5 = sinus tachycardia present? (0/1), x_6 = minimum systolic BP over 24h, x_7 = latest diastolic BP. **Labels:** $y \in \{+1$ (high risk), -1 (low risk) $\}$.

The learned tree uses only 3 out of 7 features and perfectly communicates its logic:



Reading off the rules:

1. If minimum systolic BP $< 91 \rightarrow$ High Risk.
2. If BP ≥ 91 and age $< 65 \rightarrow$ Low Risk.
3. If BP ≥ 91 and age ≥ 65 and no sinus tachycardia $\rightarrow$ Low Risk.
4. If BP ≥ 91 and age ≥ 65 and sinus tachycardia present $\rightarrow$ High Risk.

A cardiologist can verify each rule. A logistic regression weight vector cannot be audited this way.

Classifying a test patient. Suppose a new patient has $x_2 = 70$ (age), $x_5 = 1$ (sinus tachycardia present), $x_6 = 97$ (min. systolic BP). We walk the tree from the root:

1. **Root:** Is $x_6 \geq 91$? $97 \geq 91$ — **Yes** $\rightarrow$ go right.
2. **Node:** Is $x_2 \geq 65$? $70 \geq 65$ — **Yes** $\rightarrow$ go right.
3. **Node:** Is $x_5 \geq 1$? $1 \geq 1$ — **Yes** $\rightarrow$ go right.
4. **Leaf:** **High Risk (+1).**

2 A Second Example: Predicting a Cricket Match Result

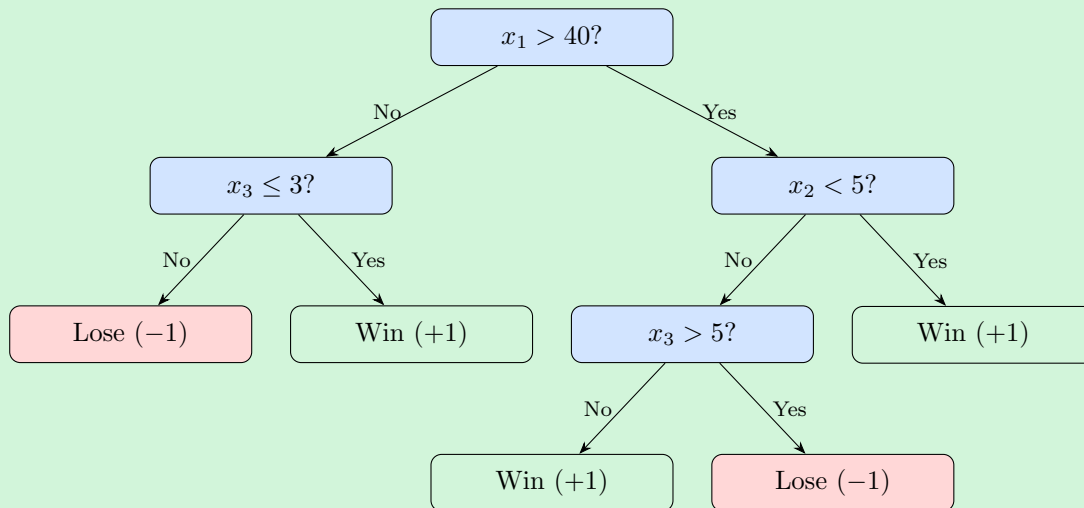
Second-Innings Run Chase — Will the Batting Team Win?

Setting: A limited-overs cricket match (50 overs per side). The batting team is in their second innings, chasing a target. We want to predict whether they will *win* ($y = 1$) or *lose/not win* ($y = 0$) based on their match situation at the end of each over.

Features:

- x_1 = runs still required (target – runs scored so far)
- x_2 = overs remaining
- x_3 = wickets lost so far (0–10)

A plausible learned tree (illustrative):



Reading off the rules:

1. If runs required ≤ 40 and ≤ 3 wickets lost $\rightarrow$ **Lose**.
2. If runs required ≤ 40 and > 3 wickets lost $\rightarrow$ **Win**.
3. If runs required > 40 and overs remaining $\geq 5 \rightarrow$ **Win**.
4. If runs required > 40 , overs remaining < 5 , and ≤ 5 wickets lost $\rightarrow$ **Win**.
5. If runs required > 40 , overs remaining < 5 , and > 5 wickets lost $\rightarrow$ **Lose**.

Classifying a test snapshot. The batting team needs 35 more runs ($x_1 = 35$), has 3 overs left ($x_2 = 3$), and has lost 5 wickets ($x_3 = 5$). Walk the tree:

1. **Root:** Is $x_1 > 40$? $35 > 40$ — **No** $\rightarrow$ go left.
2. **Node:** Is $x_3 \leq 3$? $5 \leq 3$ — **No** $\rightarrow$ go right.
3. **Leaf:** **Win** (+1).

The tree predicts a win: the target is reachable (≤ 40 runs) and enough wickets remain (> 3 still in hand).